

Tarea 2: Simulación Estocástica

Pablo León Evia

2026-09-06

1. Implementa el generador de números aleatorios (PRNG) propuesto por Wichmann & Hill en 1982, el cual se define como:

1. Implementa el generador de números aleatorios (PRNG) propuesto por Wichmann & Hill en 1982, el cual se define como:

$$\begin{aligned}X_{n+1} &= 171X_n \pmod{30269} \\Y_{n+1} &= 172Y_n \pmod{30307} \\Z_{n+1} &= 170Z_n \pmod{30323}.\end{aligned}$$

Y así definimos un intervalo unitario como:

$$U_{n+1} = \left(\frac{X_{n+1}}{30269} + \frac{Y_{n+1}}{30307} + \frac{Z_{n+1}}{30323} \right) \pmod{1}.$$

(a) Elige semillas iniciales X_0 , Y_0 , Z_0 mayores a cero y menores a 30269, 30307 y 30323, respectivamente y genera una muestra de tamaño 10, 000.

```
# Creamos una función que contenga a Xn, Yn y Zn y como salida nos dé la función Un
Un <- function(n, X0, Y0, Z0) {
  # Validaciones
  ## Para n
  if (n <= 0) { stop("El valor de n debe ser mayor a 0") }
  # Para X0
  if (X0 <= 0 | X0 >= 30269) {stop("Valor fuera de rango para X0")}
  # Para Y0
  if (Y0 <= 0 | Y0 >= 30307) {stop("Valor fuera de rango para Y0")}
  # Para XZ
  if (Z0 <= 0 | Z0 >= 30323) {stop("Valor fuera de rango para Z0")}

  # Creamos un vector de dimensión nx1
  X <- numeric(n)
  Y <- numeric(n)
  Z <- numeric(n)
  # Calculamos la entrada 1 para no introducir la semilla en el ciclo for
  X[1] <- (X0*171) %% 30269
  Y[1] <- (Y0*172) %% 30307
  Z[1] <- (Z0*170) %% 30323
```

```

for (i in 2:n) {
  X[i] <- (X[i-1]*171) %% 30269
  Y[i] <- (Y[i-1]*172) %% 30269
  Z[i] <- (Z[i-1]*170) %% 30269
}
U <- (X/30269 + Y/30307 + Z/30323)%%1
return(U)
}
muestra <- Un(n= 10000, X0 = 3253, Y0= 11782, Z0= 27453)

```

Se generó una muestra de 10,000 datos, se mostrará gráficamente en el siguiente histograma:

Histograma de la generación de 10,000 muestras uniformes(0,1)

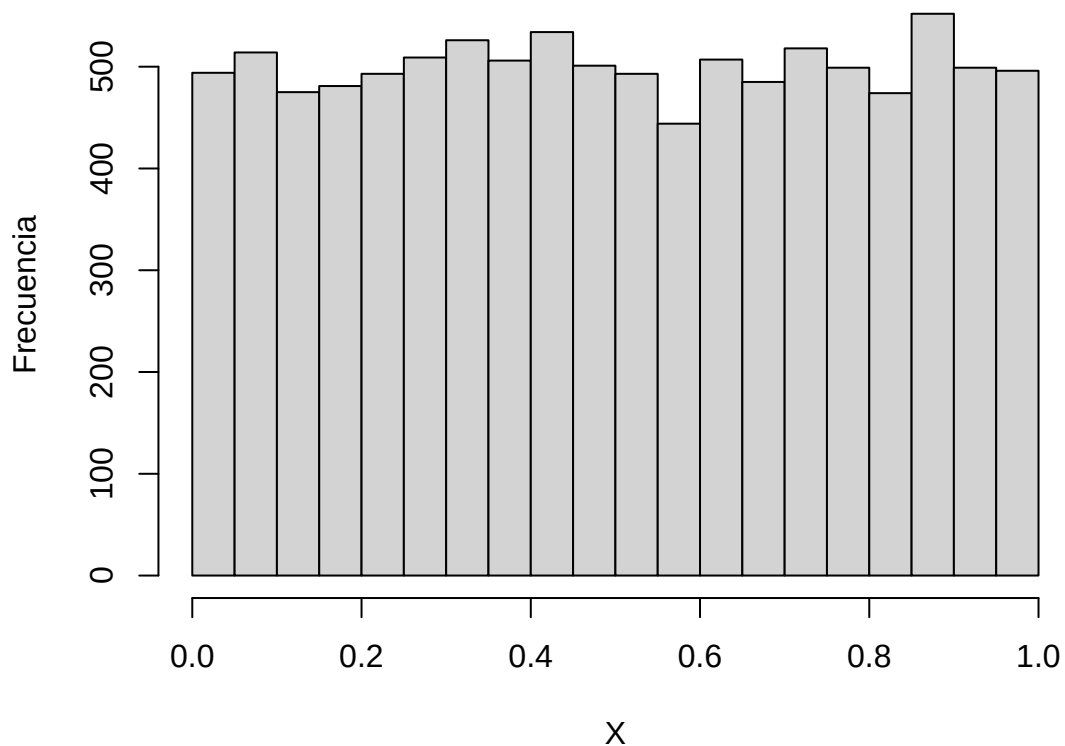


Figure 1: Figura 1

(b) Con base en las herramientas gráficas y pruebas estadísticas estudiadas en el curso, evalúa si existe evidencia de desviaciones importantes de uniformidad o independencia en la secuencia generada. Con la figura 1, podemos sospechar que existe uniformidad en los datos de la muestra generada a partir del algoritmo de Wichmann & Hill, haremos pruebas de bondad de ajuste de Kolmogorov-Smirnov para probar uniformidad y la de Ljung-Box para corroborar correlación lineal 0. # Prueba de Kolmogorov-Smirnov

```

# Evaluamos la muestra generada contra la distribución uniforme
ks.test(muestra, "punif", min = 0, max = 1)

```

```
##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  muestra
## D = 0.005632, p-value = 0.9089
## alternative hypothesis: two-sided
```

La muestra de Kolmogorov-Smirnov dice: Con evidencia estadística suficiente, no se rechaza la hipótesis de que nuestra muestra se distribuye Uniforme(0,1) con una significancia del 5%. A continuación, graficamos la función de distribución acumulativa de una uniforme(0,1) y la función de acumulación empírica para compararlas.

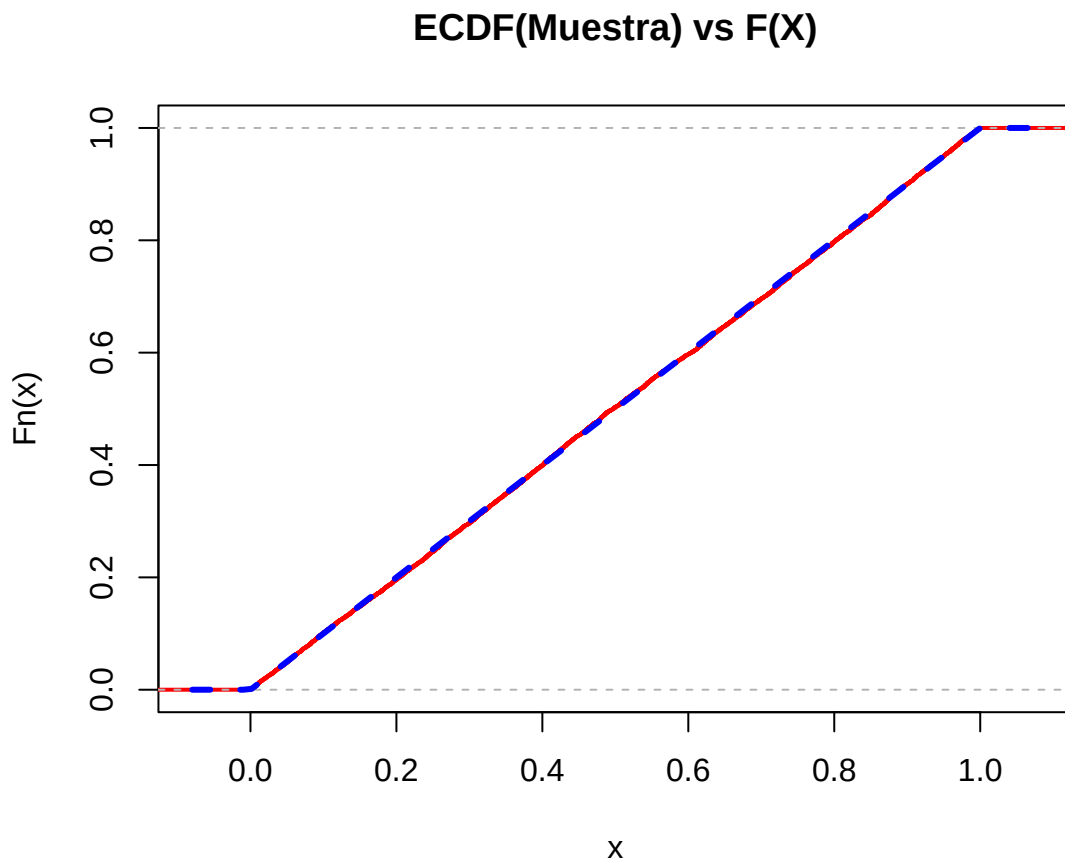


Figure 2: Figura 2

Se observa que visualmente son la misma curva.

Prueba de Ljung-Box

```
Box.test(muestra, lag = 10, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data:  muestra
## X-squared = 12.683, df = 10, p-value = 0.242
```

Se usó la prueba con un razago de los 10 datos anteriores para no perder potencia en la prueba, con un p-value del 0.242, la prueba dice: No existe suficiente evidencia estadística para rechazar la hipótesis de que la muestra tiene correlación 0. A continuación, se graficará la función de autocorrelación.

Autocorrelación de la muestra U

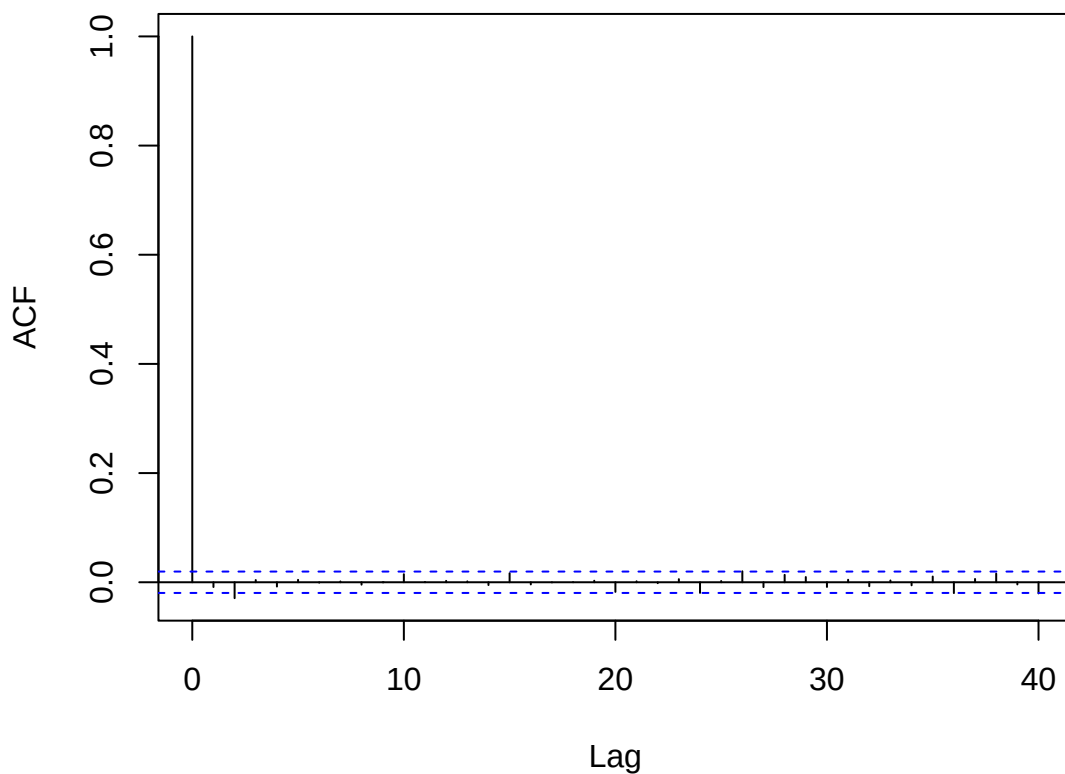


Figure 3: Figura 3

Es evidente el porque el dato X_t se relaciona consigo mismo, y muestra como no hay una recta significativa que pase de las líneas punteadas, se puede sospechar que los datos son independientes.

2. Considera la distribución Cauchy estándar cuya densidad está dada por:

$$f(x) = \frac{1}{\pi(1+x^2)}$$

(a) Encuentra la función de distribución y su inversa.

Función de distribución acumulada:

$$F(x) = \int_{-\infty}^x \frac{1}{\pi(1+t^2)} dt = \frac{1}{\pi} [\arctan(t)]_{-\infty}^x$$

$$F(x) = \frac{1}{\pi} \left(\arctan(x) - \left(-\frac{\pi}{2} \right) \right) = \frac{1}{\pi} \arctan(x) + \frac{1}{2}$$

Función de distribución inversa:

$$u = \frac{1}{\pi} \arctan(x) + \frac{1}{2}$$

$$\pi \left(u - \frac{1}{2} \right) = \arctan(x)$$

$$F^{-1}(u) = \tan \left(\pi \left(u - \frac{1}{2} \right) \right)$$

Diseña un algoritmo para simular variables aleatorias Cauchy estándar e implementalo. A partir de la función inversa podemos generar valores aleatorios provenientes de la distribución Cauchy, a partir de una Uniforme(0,1), como la imagen de la inversa está en el conjunto (0,1).

```
# Definimos la función para generar puntos Cauchy
set.seed(14214)
Cauchy <- function(n) {
  vector <- numeric(n)
  vector <- runif(n)
  Cau_point <- tan(pi*(vector-0.5))

  return(Cau_point)
}
hist(Cauchy(100), freq = FALSE, xlim = c(-10, 10), breaks = 100,
     main = "f(x) vs Histograma", xlab = "x")
curve(dcauchy(x), col = "red", add = TRUE)
```

```
set.seed(14214)
```

- (c) Genera 100 muestras de tamaños 100, 1000, 10000 y 100000 y analiza el comportamiento de $\bar{x}$. Explica el comportamiento observado a partir de las propiedades de la distribución Cauchy.*

Crearemos una función que tenga anidada la Función Cauchy que creamos, nos dará el resultado de la media, es claro que no puedo establecer una semilla para que sean N muestras aleatorias de tamaño n.

```
# Por default está el 100 como cantidad de muestras.
cauchy_mean <- function(N=100,n){
  x_mean <- numeric(N)
  for (i in 1:N) {
    vector <- Cauchy(n)
    x_mean[i] <- mean(vector)
  }

  print(summary(x_mean))
  hist(x_mean, main = paste("Histograma de media de una cauchy de tamaño:", n), breaks = 25)

  return(x_mean)
}
```

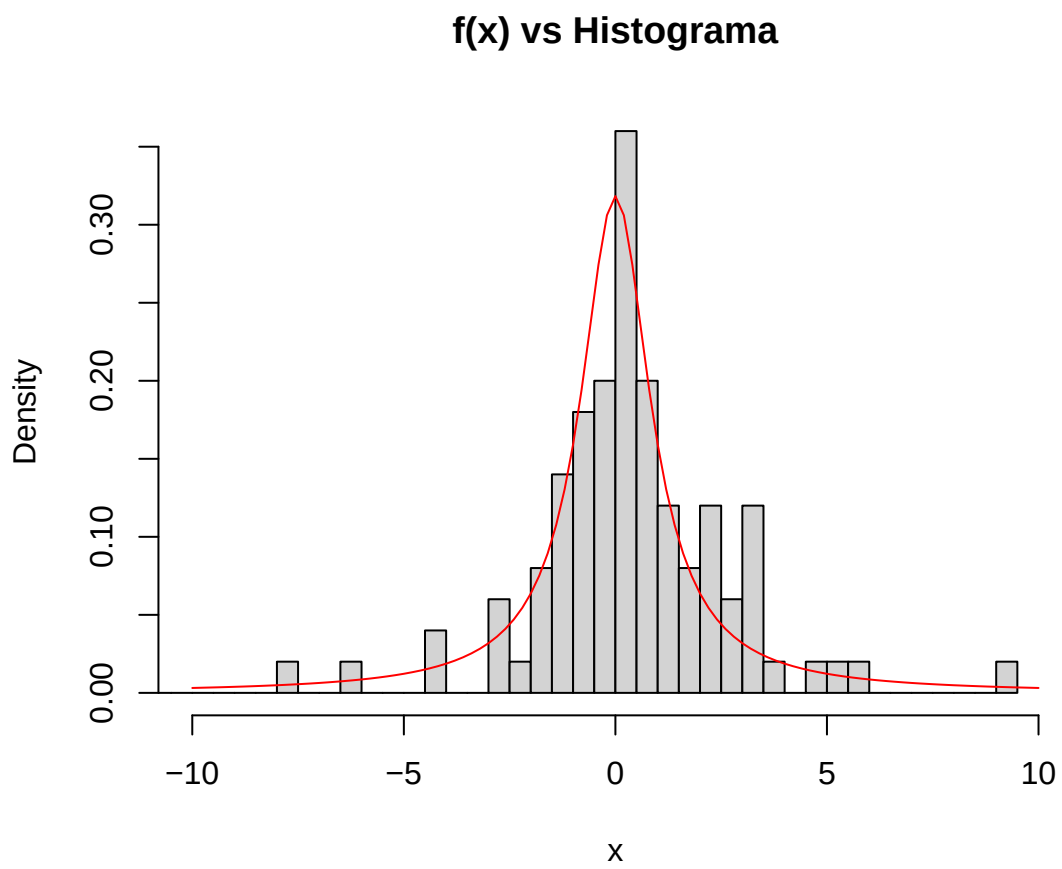


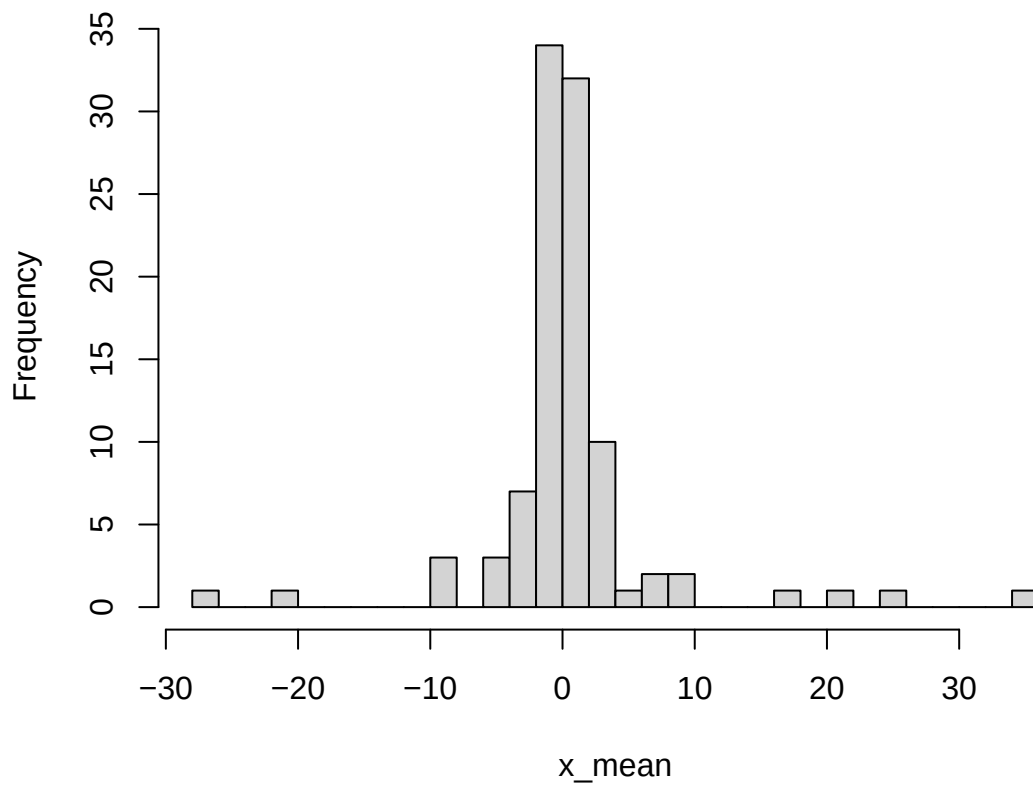
Figure 4: Figura 4

La función anterior nos dará un resumen estadístico sobre las “medias” de cada muestra y nos graficará un histograma a partir de las medias, nos automatiza para poder solo llamar la función con cada tipo de caso requerido.

```
# Evaluamos los 4 tamaños de muestra pedidos
x_100 <- cauchy_mean(n = 100)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## -27.70862 -1.00339  0.01061  0.44122  1.05243  35.23102
```

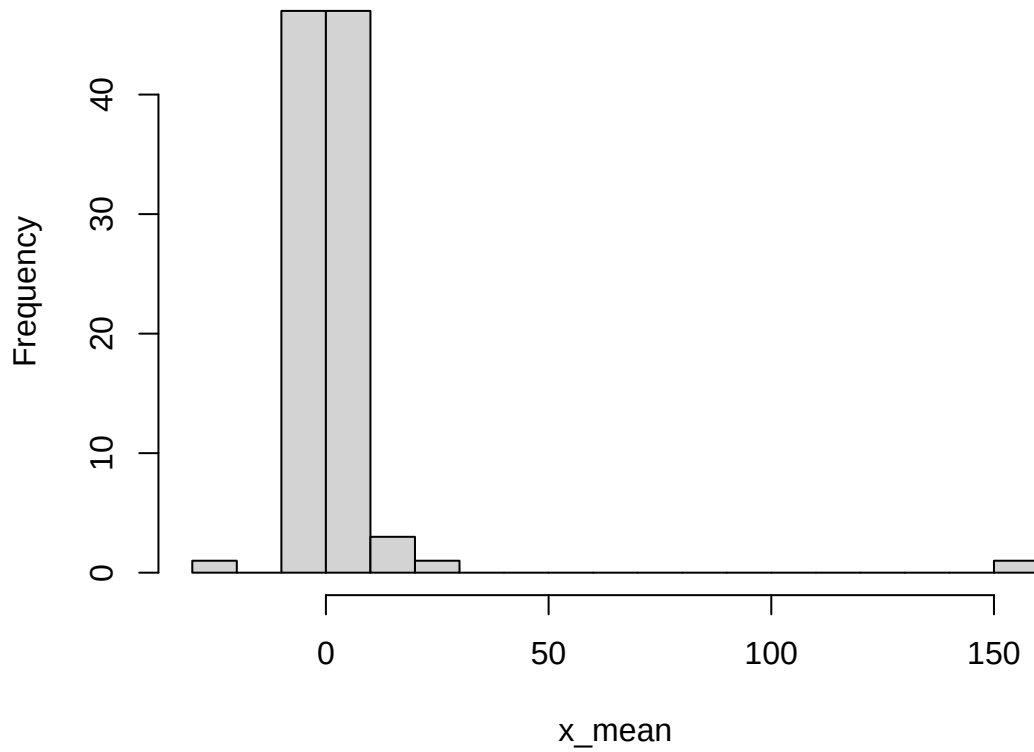
Histograma de media de una cauchy de tamaño: 100



```
x_1000 <- cauchy_mean(n = 1000)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## -29.2383 -0.7362  0.1254  1.6739  0.7325 153.5419
```

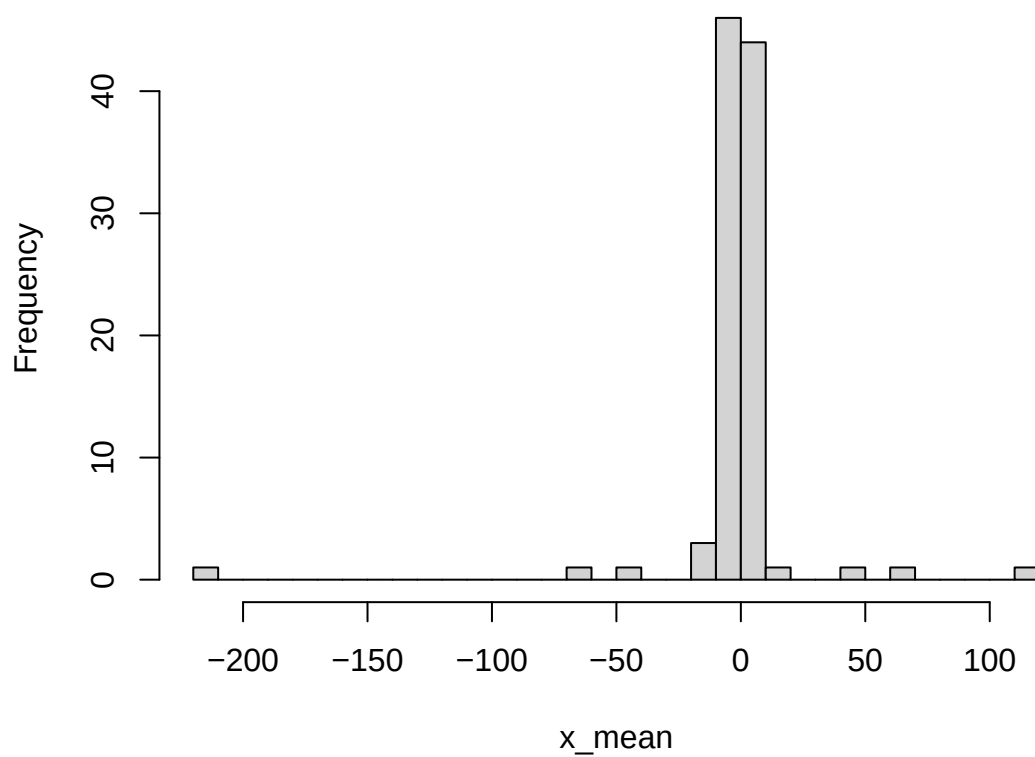
Histograma de media de una cauchy de tamaño: 1000



```
x_10000 <- cauchy_mean(n = 10000)
```

```
##      Min.  1st Qu.  Median    Mean  3rd Qu.    Max.
## -211.9839 -1.1345  -0.0501  -1.1407   1.2179  119.7456
```

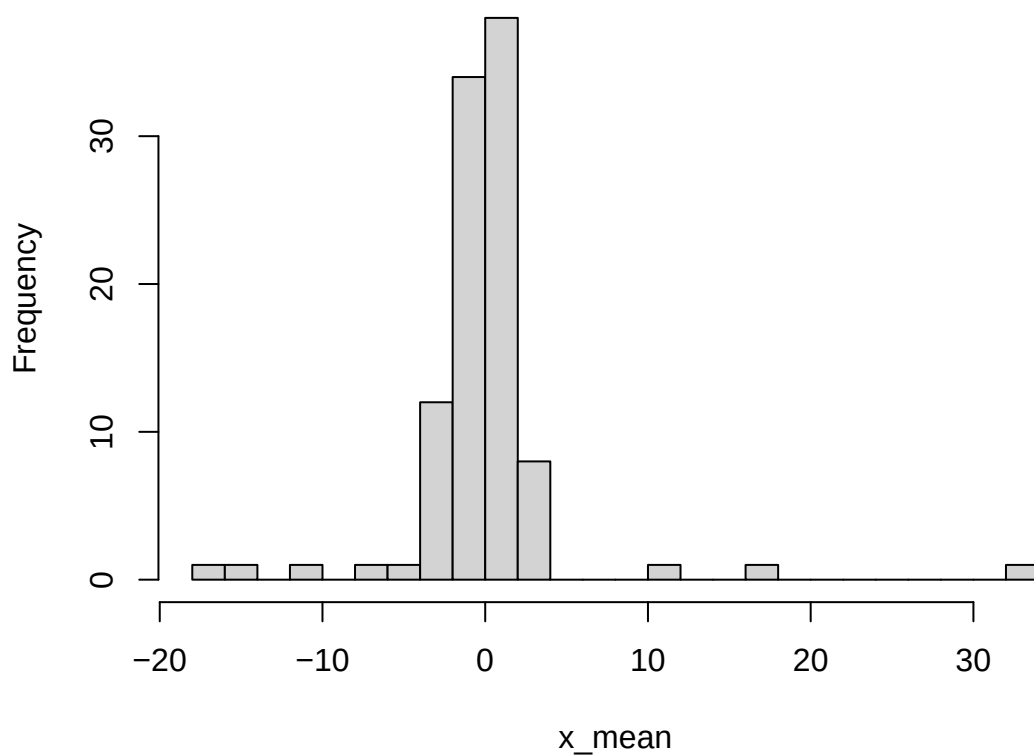
Histograma de media de una cauchy de tamaño: 10000



```
x_100000 <- cauchy_mean(n = 100000)
```

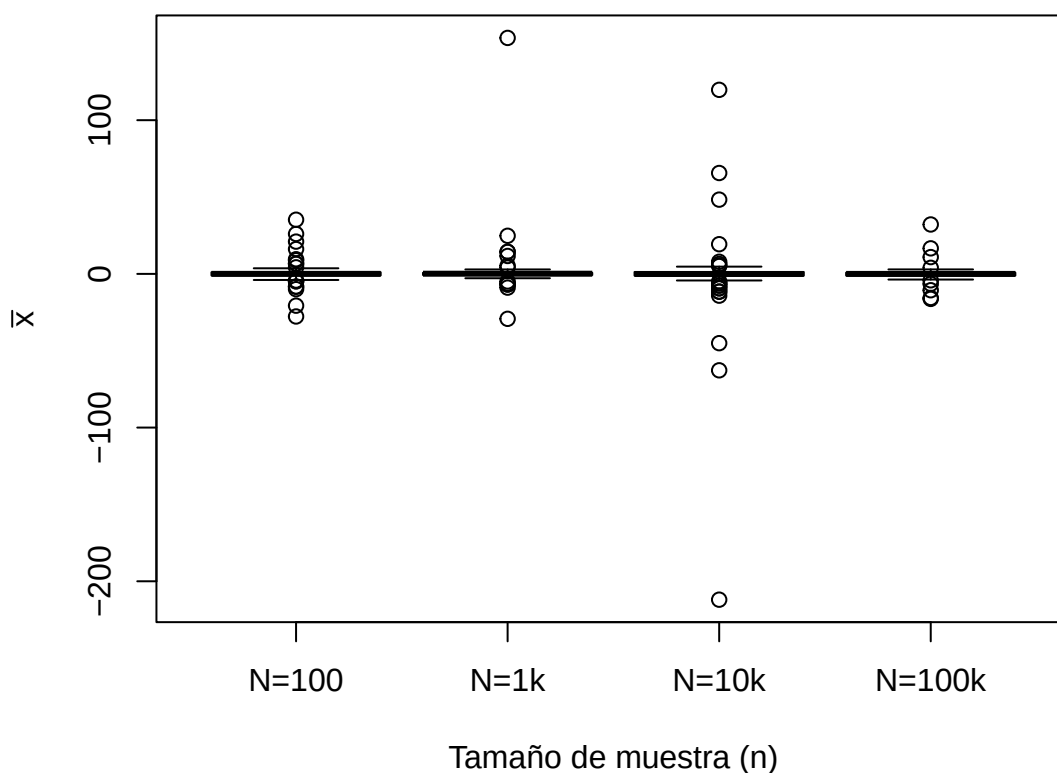
```
##      Min.    1st Qu.    Median      Mean    3rd Qu.     Max.
## -16.271823 -1.085723 -0.027859  0.000763  0.810043  32.157293
```

Histograma de media de una cauchy de tamaño: 1e+05



```
# Gráfico comparativo de la variabilidad
boxplot(list("N=100" = x_100, "N=1k" = x_1000, "N=10k" = x_10000, "N=100k" = x_100000),
  main = expression("Comportamiento de " ~ bar(x) ~ " según " ~ n),
  xlab = "Tamaño de muestra (n)",
  ylab = expression(bar(x)),
  col = "lightblue")
```

Comportamiento de $\bar{x}$ según n



Los valores que toman los gráficos son pseudoaleatorios debido a que no hay una semilla establecida (sería interesante incluir un `set.seed(### + 1)` previo para garantizar la reproducibilidad). Por otro lado, la distribución Cauchy no tiene ningún momento definido debido a que su integral no es absolutamente convergente y es de cola pesada (heavy tail), lo que genera valores atípicos (outliers) con alta frecuencia. Debido a esto, la esperanza y la varianza no se pueden aproximar empíricamente mediante simulación (el promedio muestral no converge); aparte, no cumple el teorema de Los Grandes Números ni el Teorema del Límite Central.

3. Simulación de una variable aleatoria $Poisson(\lambda)$

(a) Diseña e implementa un algoritmo utilizando la función inversa generalizada y evaluando de forma exacta la función de masa, esto es:

$$\mathbb{P}(X = k) = \frac{\lambda^k \exp(-\lambda)}{k!}.$$

```
poisson_version1 <- function(n, lambda) {
  muestras <- numeric(n)

  for (i in 1:n) {
    unif <- runif(1)
    k <- 0
```

```

    # Avanza por las particiones F(k) hasta que la uniforme caiga dentro
    while (unif > ppois(k, lambda)) {
      k <- k + 1
    }

    muestras[i] <- k
  }

  return(muestras)
}
poisson_version1(n=10, lambda = 50)

```

```
## [1] 49 59 60 54 52 50 48 53 50 47
```

La idea del algoritmo anterior es que vaya generando un punto uniforme y la función while es la que nos genera las particiones y justamente el ancho de la partición, lo que por probabilidad clásica es más probable que caiga en números cercanos a la media. (b) *Genera una segunda versión del algoritmo utilizando la propiedad recursiva de la función masa.* A continuación, aprovecharemos la siguiente propiedad de la función Poisson:

$$p_k = \mathbb{P}(X = k) = \frac{\lambda^k e^{-\lambda}}{k!} = \frac{\lambda}{k} \cdot \frac{\lambda^{k-1} e^{-\lambda}}{(k-1)!} = \frac{\lambda}{k} \cdot p_{k-1}$$

```

poisson_version2 <- function(n, lambda) {
  muestras <- numeric(n)

  for (i in 1:n) {
    unif <- runif(1)
    k <- 0 # Valor de la v.a poi
    p <- exp(-lambda) # Caso P(X=0)
    Fx <- p # Cota superior de la partición

    # Avanza por las particiones F(k) hasta que la uniforme caiga dentro
    while (unif > Fx) {
      k <- k + 1
      # Aprovechamos la prop. recursiva
      p <- lambda * p / k
      # Para que no se detenga solo
      Fx <- Fx + p
    }

    muestras[i] <- k
  }

  return(muestras)
}
poisson_version2(n=10, lambda = 513)

```

```
## [1] 513 452 505 511 504 513 516 510 468 487
```

Lo mencionado anteriormente, los valores generados están cerca de su media, pero de manera recursiva se calcularon las particiones, lo que vuelve más claro lo que se realiza enés de requerir de la función “ppois()” que contiene R.

(d) Verifica mediante herramientas gráficas y estadísticas apropiadas que los algoritmos producen muestras compatibles con una distribución $\text{Poisson}(\lambda)$. Sabemos que un proceso de Poisson se relaciona con una densidad $\text{Pois}(t\lambda)$. Generaremos las exponenciales con la función inversa, lo unico en lo que usaremos r para generar puntos es en la uniforme, la función de la exponencial es el siguiente:

```
exponencial <- function(n, r){
  u <- runif(n)
  x <- -log(u) / r
  return(x)
}
```

A continuación, responderemos la pregunta 3d:

```
poisson_version3 <- function(n, lambda) {
  muestras <- numeric(n)

  for (i in 1:n) {
    tiempo_acumulado <- 0
    k <- 0

    # Generea exponenciales apartir de la función creada arriba
    while (tiempo_acumulado <= 1) {
      t_i <- exponencial(1, r = lambda)
      tiempo_acumulado <- tiempo_acumulado + t_i
      k <- k + 1
    }

    # Descontamos 1 porque el último evento sobrepasó el tiempo 1
    muestras[i] <- k - 1
  }

  return(muestras)
}

# Prueba de ejecución
poisson_version3(n = 10, lambda = 50)
```

```
## [1] 58 49 44 50 38 57 47 58 54 51
```

(e) Compara el tiempo de ejecución de los tres algoritmos para generar muestras de tamaños $n = 100, 1000, 10000, 100000$ y para $\lambda = 1, 10, 50$. ¿Cuál resulta más eficiente en cada caso? Discute los resultados considerando tanto el tiempo de cómputo como las operaciones requeridas por cada algoritmo. `library(microbenchmark)` `library(dplyr)` `library(purrr)` Usaremos tres paqueterías para que no se extienda el documento, primero `microbenchmark` que es para correr simultáneamente las comparaciones de los tres métodos, `dplyr` se usó para la función `transmute` que es similar a `mutate` pero está desecha las columnas que no queremos; la paquetería `purrr` se usó para aplicar una función repetidamente a cada elemento de una lista, un vector o las filas/columnas de un data frame sin necesidad de escribir bucles `for` y une los resultados en formato Data Frame por filas.

```
# Valores a evaluar
parametros <- expand.grid(
  n = c(100, 1000, 10000, 100000),
  lambda = c(1, 10, 50)
```

```

)

# Benchmark para que calculo de un solo paso
resultados_benchmark <- map_dfr(1:nrow(parametros), function(i) {
  n_val <- parametros$n[i]
  lam_val <- parametros$lambda[i]

  # Medimos los 3 algoritmos para la combinación actual
  bench <- microbenchmark(
    V1_Inversa = poisson_version1(n_val, lam_val),
    V2_Recursiva = poisson_version2(n_val, lam_val),
    V3_Proceso = poisson_version3(n_val, lam_val),
    times = 5, # Número de repeticiones por combinación
    unit = "s" # Tiempos en segundos
  )

  # Resumimos la mediana de tiempo en segundos
  summary(bench) %>%
  as.data.frame() %>%
  transmute(
    n = n_val,
    lambda = lam_val,
    Algoritmo = expr,
    Tiempo_Mediano_Seg = median
  )
})

# 3. Visualizamos la tabla consolidada
print(resultados_benchmark)

```

##	n	lambda	Algoritmo	Tiempo_Mediano_Seg
## 1	1e+02	1	V1_Inversa	0.000159201
## 2	1e+02	1	V2_Recursiva	0.000058601
## 3	1e+02	1	V3_Proceso	0.000164801
## 4	1e+03	1	V1_Inversa	0.001703601
## 5	1e+03	1	V2_Recursiva	0.000549001
## 6	1e+03	1	V3_Proceso	0.001541400
## 7	1e+04	1	V1_Inversa	0.017319801
## 8	1e+04	1	V2_Recursiva	0.005535701
## 9	1e+04	1	V3_Proceso	0.016703002
## 10	1e+05	1	V1_Inversa	0.176127800
## 11	1e+05	1	V2_Recursiva	0.055870901
## 12	1e+05	1	V3_Proceso	0.165195001
## 13	1e+02	10	V1_Inversa	0.000756601
## 14	1e+02	10	V2_Recursiva	0.000093302
## 15	1e+02	10	V3_Proceso	0.000812001
## 16	1e+03	10	V1_Inversa	0.007939201
## 17	1e+03	10	V2_Recursiva	0.000914700
## 18	1e+03	10	V3_Proceso	0.008721101
## 19	1e+04	10	V1_Inversa	0.080548801
## 20	1e+04	10	V2_Recursiva	0.009344301
## 21	1e+04	10	V3_Proceso	0.092638301
## 22	1e+05	10	V1_Inversa	0.791923001

## 23	1e+05	10	V2_Rekursiva	0.092558702
## 24	1e+05	10	V3_Proceso	0.892127900
## 25	1e+02	50	V1_Inversa	0.003918201
## 26	1e+02	50	V2_Rekursiva	0.000265702
## 27	1e+02	50	V3_Proceso	0.003712701
## 28	1e+03	50	V1_Inversa	0.036812702
## 29	1e+03	50	V2_Rekursiva	0.002503800
## 30	1e+03	50	V3_Proceso	0.040544901
## 31	1e+04	50	V1_Inversa	0.369073702
## 32	1e+04	50	V2_Rekursiva	0.026095202
## 33	1e+04	50	V3_Proceso	0.402770101
## 34	1e+05	50	V1_Inversa	3.673690102
## 35	1e+05	50	V2_Rekursiva	0.253818801
## 36	1e+05	50	V3_Proceso	4.043070202

Para todas las situaciones, ganó la función recursiva, como segundo lugar quedó el de la inversa, hay una clara conexión entre que la función inversa haya quedado segundo y el método como Proceso de Poisson tercero, debido a que internamente el proceso Poisson aplica el método de la función inversa. Es claro que el de menor calculo e iteraciones es el segundo método.